

THIẾT KẾ CHI TIẾT — CHECKOUT SERVICE (ORCHESTRATOR)

Khung kiến trúc: msfw Hexagonal | Domain Context: checkout | Phiên bản: v1.0

1. Cấu trúc thành phần Module (Hexagonal msfw Layout)

Dịch vụ Checkout đóng vai trò là Nhạc trưởng (Saga Orchestrator) đồng bộ, điều phối quy trình mua bán phức tạp xuyên miền mà bản thân nó không sở hữu database quan hệ, duy trì phiên tạm qua hạ tầng Redis tốc độ cao.

ADAPTER LAYER (Vỏ ngoài)	APPLICATION LAYER (Điều phối)	DOMAIN LAYER (Lõi nghiệp vụ)
inbound - CheckoutController (REST API) outbound - CheckoutSession0a (Redis Client) - CatalogClient0a (gRPC Client) - InventoryClient0a (gRPC Client) - OrderClient0a (gRPC Client)	ports.in (Use Cases) - SubmitCheckout ports.out (Cổng gọi ra) - CatalogPort, InventoryPort - OrderPort, PaymentPort	models - CheckoutSaga (Aggregate Root) - IdempotencyKey (Identity) - CartSnapshot (Value Object)

2. Danh sách Invariants (Bất biến nghiệp vụ) của Domain

- Bất biến 1: Tiến trình Saga phân tán chạy tuyến tính một chiều:** Trạng thái của phiên giao dịch điều phối (Saga State) bắt buộc phải dịch chuyển tuần tự qua bộ lọc in-memory nghiêm ngặt: **PRICING ightarrow RESERVING ightarrow ORDERING ightarrow ESCROWING ightarrow REDIRECTED**. Nghiêm cấm hành vi nhảy bước khi các dịch vụ phụ thuộc chưa phản hồi kết quả thành công.
- Bất biến 2: Thứ tự bù đắp dữ liệu đảo ngược (Strict Reverse-Order Compensation):** Khi bất kỳ một mắt xích nào trong chuỗi Saga đồng bộ báo lỗi (ví dụ bước gọi khởi tạo quỹ tiền trung gian Escrow thất bại), thực thể CheckoutSaga bắt buộc phải kích hoạt chuỗi hành động rollback nghiệp vụ theo ****thứ tự đảo ngược chính xác**** so với các bước đã thành công trước đó (Hủy lệnh đặt hàng tại Order Svc trước, sau đó mới giải phóng khóa hàng tại Inventory Svc), triệt tiêu hoàn toàn rủi ro xuất hiện đơn hàng mồ côi hoặc kho bị treo vĩnh viễn (No ghost reservations allowed).
- Bất biến 3: Thẩm quyền tối cao của Giá thành gốc (Price Authority Invariant):** Dữ liệu giá của các SKU trong giỏ hàng bắt buộc phải lấy trực tiếp từ cổng kết nối CatalogPort phát sinh tại thời điểm chạy luồng

điều phối. Toàn bộ thông tin về giá do phía thiết bị client truyền lên trong request body bắt buộc phải bị loại bỏ để chống rủi ro thao túng giá thành.

- **Bất biến 4: Khóa khử trùng phiên trùng lặp cố định qua Redis:** Mã định danh IdempotencyKey truyền lên từ header bắt buộc phải được bền vững hóa tạm thời vào Redis dưới dạng khóa phân tán (Distributed Lock) với thời gian khoá tối đa là 10 giây để chống hiện tượng Buyer ấn double-submit do trễ mạng mạng, đồng thời lưu trữ kết quả phiên thành công trong vòng 30 phút.

3. Thiết kế chi tiết Interface Use Cases & Outbound Ports

Khai báo Use Case điều phối và hệ thống Cổng đầu ra của tầng Application dịch vụ Checkout:

```
package tech.vsf.ptnt.msfw.checkout.application.port.in;

import tech.vsf.ptnt.msfw.checkout.application.dto.*;

// 3.1 Inbound Port: Tiếp nhận yêu cầu thanh toán giỏ hàng đa thương nhân toàn cục
public interface SubmitCheckout {
    CheckoutResultView execute(SubmitCheckoutCmd cmd);
}

// --- Data Contracts ---
public record SubmitCheckoutCmd(
    String idempotencyKey, // Nhận từ HTTP Header X-Idempotency-Key
    String buyerId,
    ShippingAddressInput address,
    List items
) {}

public record CheckoutItemInput(
    String sku,
    int quantity,
    String merchantId
) {}

public record CheckoutResultView(
    String paymentUrl,
    List orderIds,
    Long grandTotalAmount
) {}
```

Hệ thống Cổng đầu ra (Outbound Ports) do tầng Application Checkout định nghĩa để tầng Adapter cài đặt cấu hình gRPC Client:

```
package tech.vsf.ptnt.msfw.checkout.application.port.out;

import tech.vsf.ptnt.msfw.checkout.application.dto.*;
import java.util.List;

public interface CatalogPort {
    List fetchPrices(List skuCodes);
}

public interface InventoryPort {
    ReservationDto reserveStock(String orderRef, List items);
    void releaseStock(String reservationId);
}

public interface OrderPort {
    String createPendingOrder(CreateOrderDto dto);
    void cancelOrder(String orderId, String reason);
}

public interface PaymentPort {
    String initEscrow(Long totalAmount, List orderIds, String buyerId);
}
```